

Implementation and Optimization of the HQC Decoder on NPU-Integrated Devices

No Institute Given

Abstract. Recently, the U.S. National Institute of Standards and Technology (NIST) standardized a set of post-quantum cryptographic algorithms to protect digital communications against future quantum attacks. To provide an additional line of defense for general-purpose encryption, NIST later selected HQC as the fifth post-quantum encryption algorithm and as a backup to ML-KEM. In this work, we focus on implementing and optimizing the decoding operation of HQC on Qualcomm Hexagon processors. In particular, we exploit the Hexagon Vector eXtensions (HVX) units to process element-wise operations in the Reed-Muller and Reed-Solomon decoders in parallel, thereby reducing energy consumption and overall processor-cycle (P-cycle) runtime. We decompose the decoding operation into multiple modules and describe the adaptations tailored to NPU-integrated devices.

1 Introduction

Post-quantum cryptography has been widely studied in recent years, since classical public-key cryptographic primitives based on integer factorization and discrete logarithms are vulnerable to large-scale quantum attacks. To address this threat, the U.S. National Institute of Standards and Technology (NIST) initiated a process to standardize quantum-resistant public-key cryptographic algorithms. After approving the first post-quantum cryptographic standards in 2024, including ML-KEM, ML-DSA, and SLH-DSA [5,4,6], NIST selected HQC as an additional code-based key encapsulation mechanism and as a backup to ML-KEM [7].

HQC relies on error-correcting codes and provides algorithmic diversity compared with lattice-based schemes. Its decapsulation procedure contains several decoding components, including Reed-Muller and Reed-Solomon decoding, which involve many regular arithmetic and data-rearrangement operations. These operations expose substantial data-level parallelism and are therefore suitable for vectorized implementation.

In this work, we target Qualcomm Hexagon processors and optimize the HQC decoding operation by exploiting the hardware acceleration features of the Hexagon architecture. In particular, we leverage the Hexagon Vector eXtensions (HVX), a wide-SIMD vector extension, to process multiple symbols or coefficients in parallel [8]. We decompose the decoder into several modules and adapt the Reed-Muller and Reed-Solomon decoding steps to HVX-friendly execution patterns, including element-wise arithmetic, comparisons, vector rearrangements, and Hadamard-transform stages. We also discuss the Hexagon Matrix eXtension (HMX) as part of the broader Hexagon acceleration architecture [10], while showing that HQC decoding is primarily suited to HVX-style vector parallelism. Our optimized implementation aims to reduce both processor-cycle runtime and energy consumption on NPU-integrated Qualcomm devices.

Our contribution:

2 Background

2.1 Notation

Let $\mathbb{Z}$ denote the ring of integers, $\mathbb{F}_2$ denote the binary finite field and $\mathcal{B}$ denote the set $\{0, \dots, 255\}$ of unsigned 8-bit integers. Given a set S , let $s \xleftarrow{\$} S$ denote that s is chosen uniformly at random from S . Let $\mathcal{V}$ denote a vector space of dimension n over $\mathbb{F}_2$ for some positive $n \in \mathbb{Z}$. For a linear code $\mathcal{C}$ of length n , dimension k and distance d , we denote as $[n, k, d]$

2.2 Reed-Muller code

Let m and r be non-negative integers with $0 \leq r \leq m$. The binary Reed-Muller code of order r and length 2^m , denoted by $\text{RM}(r, m)$, is defined as the set of evaluation vectors of all Boolean polynomials of degree at most r over $\mathbb{F}_2^m$. More precisely, let $\mathcal{P}_{r,m}$ be the set of polynomials.

$$f(x_1, \dots, x_m) = \sum_{\substack{I \subseteq \{1, \dots, m\} \\ |I| \leq r}} a_I \prod_{i \in I} x_i, \quad a_I \in \mathbb{F}_2.$$

Then

$$\text{RM}(r, m) = \left\{ (f(\mathbf{x}))_{\mathbf{x} \in \mathbb{F}_2^m} : f \in \mathcal{P}_{r,m} \right\} \subseteq \mathbb{F}_2^{2^m}.$$

The parameters of $\text{RM}(r, m)$ are

$$n = 2^m, \quad k = \sum_{i=0}^r \binom{m}{i}, \quad d = 2^{m-r},$$

where n is the code length, k is the dimension, and d is the minimum Hamming distance.

In particular, the first-order Reed-Muller code $\text{RM}(1, m)$ consists of the evaluations of all affine Boolean functions

$$f_{\mathbf{a},b}(\mathbf{x}) = b + \langle \mathbf{a}, \mathbf{x} \rangle, \quad \mathbf{a} \in \mathbb{F}_2^m, \quad b \in \mathbb{F}_2,$$

where $\langle \mathbf{a}, \mathbf{x} \rangle = \sum_{i=1}^m a_i x_i \bmod 2$. Hence, $\text{RM}(1, m)$ has parameters $[2^m, m+1, 2^{m-1}]_2$.

Decoding first-order Reed-Muller codes can be performed efficiently using the fast Hadamard transform. Given a received vector $\mathbf{y} = (y_{\mathbf{x}})_{\mathbf{x} \in \mathbb{F}_2^m}$, one first maps its binary entries to signs i.e $z_{\mathbf{x}} = (-1)^{y_{\mathbf{x}}}$. For each $\mathbf{a} \in \mathbb{F}_2^m$, the decoder computes the correlation

$$W(\mathbf{a}) = \sum_{\mathbf{x} \in \mathbb{F}_2^m} z_{\mathbf{x}} (-1)^{\langle \mathbf{a}, \mathbf{x} \rangle}.$$

The value of $\mathbf{a}$ maximizing $|W(\mathbf{a})|$ determines the linear part of the decoded affine function, while the sign of $W(\mathbf{a})$ determines the constant term b .

2.3 Reed-Solomon codes

Reed-Solomon (RS) codes are a family of non-binary error-correcting codes defined over a finite field. Let $\mathbb{F}_q$ be a finite field with q elements, and let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ be distinct evaluation points, where $n \leq q$. For a message polynomial $m(X) \in \mathbb{F}_q[X]$ of degree less than k , the corresponding Reed-Solomon codeword is obtained by evaluating $m(X)$ at these points:

$$\text{Enc}(m) = (m(\alpha_1), m(\alpha_2), \dots, m(\alpha_n)) \in \mathbb{F}_q^n.$$

The Reed-Solomon code of length n and dimension k is therefore defined as

$$\text{RS}_q(n, k) = \left\{ (m(\alpha_1), \dots, m(\alpha_n)) : m(X) \in \mathbb{F}_q[X], \deg(m) < k \right\}.$$

where $d = n - k + 1$, where d is the minimum Hamming distance.

For decoding, let $\mathbf{r} = \mathbf{c} + \mathbf{e}$ be the received word, where $\mathbf{c}$ is the transmitted codeword and $\mathbf{e}$ is the error vector. The decoder first computes a syndrome vector that depends only on the error pattern. In the cyclic formulation of Reed-Solomon codes, the syndromes are typically computed as

$$S_j = r(\alpha^j), \quad j = 1, \dots, 2t,$$

where α is a primitive element of $\mathbb{F}_q$. If all syndromes are zero, the received word is already a valid codeword. Otherwise, the decoder solves the key equation to find the error-locator polynomial

$$\Lambda(X) = \prod_{i \in \mathcal{E}} (1 - X\alpha^i),$$

where $\mathcal{E}$ denotes the set of error positions. The roots of $\Lambda(X)$ identify the error locations, and the corresponding error values are then recovered using the error-evaluator polynomial, for example through Forney's formula.

2.4 HQC

HQC relies on concatenated code where the inner and outer codes are ML and RS, respectively. Specifically, for HQC, it contains an external code $[n_e, k_e, d_e]$ over $\mathbb{F}_q$ and an internal code $[n_i, k_i, d_i]$ over $\mathbb{F}_2$, where $q = 2^{k_i}$. There is a one-to-one map from elements of $\mathbb{F}_q$ to the words of the internal code:

$$\mathbb{F}_q^{n_e} \rightarrow \mathbb{F}_2^N$$

where $N = n_e n_i$. The external code is thus transformed into a binary code of parameters $[N = n_e n_i, K = k_e k_i, D \geq d_e d_i]$.

HQC use shortened RS and duplicated RM codes. The parameters are listed in table 1

Instance	Reed-Muller Code	Multiplicity	Duplicated Reed-Muller Code
HQC-1	[128, 8, 64]	3	[384, 8, 192]
HQC-3	[128, 8, 64]	5	[640, 8, 320]
HQC-5	[128, 8, 64]	5	[640, 8, 320]

Table 1. Duplicated Reed-Muller codes.

Note. The multiplicity indicates how many times each Reed-Muller codeword is duplicated.

2.5 NPU

NPU stands for *Neural Processing Unit*. In this work, the term denotes the data-parallel execution model exposed by vector accelerators such as Qualcomm Hexagon HVX [9]. Rather than describing the optimized decoder only by processor-specific instruction names, we model the accelerator as a finite-lane vector machine. This abstraction is sufficient for the RM and RS optimizations because their dominant operations are regular lane-wise arithmetic, finite-field products, permutations, and reductions.

We mainly focus on the HQC-128 parameter set and the same optimization principles also apply to the remaining parameter sets, with changes in block sizes and the number of required vector blocks. Let L be the number of lanes in one vector register and let

$$\mathbf{x} = (x_0, \dots, x_{L-1}) \in \mathcal{R}^L.$$

The ring $\mathcal{R}$ depends on the stage being accelerated: signed 16-bit integers are used for Reed-Muller soft values, while elements of $\mathbb{F}_{2^8}$ are embedded in 16-bit lanes for Reed-Solomon arithmetic. We use the following abstract vector operations:

$$\begin{aligned} \text{VADD}(\mathbf{x}, \mathbf{y}) &= (x_i + y_i)_{i=0}^{L-1}, \\ \text{VSUB}(\mathbf{x}, \mathbf{y}) &= (x_i - y_i)_{i=0}^{L-1}, \\ \text{VABS}(\mathbf{x}) &= (|x_i|)_{i=0}^{L-1}, \\ \text{VMAX}(\mathbf{x}, \mathbf{y}) &= (\max(x_i, y_i))_{i=0}^{L-1}, \\ \text{VMIN}(\mathbf{x}, \mathbf{y}) &= (\min(x_i, y_i))_{i=0}^{L-1}. \end{aligned}$$

The operation

$$\text{VSPLAT}(a) = (a, \dots, a)$$

broadcasts a scalar to all lanes. Bit-level vector operations are represented as

$$\begin{aligned} \text{VXOR}(\mathbf{x}, \mathbf{y}) &= (x_i \oplus y_i)_{i=0}^{L-1}, \\ \text{VAND}(\mathbf{x}, \mathbf{y}) &= (x_i \wedge y_i)_{i=0}^{L-1}, \\ \text{VSHL}_s(\mathbf{x}) &= (x_i \ll s)_{i=0}^{L-1}, \\ \text{VSHR}_s(\mathbf{x}) &= (x_i \gg s)_{i=0}^{L-1}. \end{aligned}$$

These operations are used to express vectorized multiplication in $\mathbb{F}_{2^8}$, where addition is XOR and multiplication is implemented through repeated multiplication by X modulo the HQC field polynomial

$$P(X) = X^8 + X^4 + X^3 + X^2 + 1.$$

Let $\rho = 0x1d$ be the corresponding low-byte reduction constant. For a vector $\mathbf{x}$ whose lanes contain field elements embedded in 16-bit lanes, vector multiplication by X is abstracted as

$$\begin{aligned} \mathbf{c}(\mathbf{x}) &= \text{VAND}(\text{VSHR}_7(\mathbf{x}), \text{VSPLAT}(1)), \\ \mathbf{m}(\mathbf{x}) &= \text{VSUB}(\text{VSPLAT}(0), \mathbf{c}(\mathbf{x})), \\ \text{GFXTIME}(\mathbf{x}) &= \text{VAND}(\text{VXOR}(\text{VSHL}_1(\mathbf{x}), \text{VAND}(\mathbf{m}(\mathbf{x}), \text{VSPLAT}(\rho))), \text{VSPLAT}(0x\text{ff})). \end{aligned}$$

This corresponds to lane-wise multiplication by X in $\mathbb{F}_{2^8}$.

For $a \in \mathbb{F}_{2^8}$ and $\mathbf{b} = (b_0, \dots, b_{L-1}) \in \mathbb{F}_{2^8}^L$, define scalar-by-vector finite-field multiplication as

$$\text{GFVECMUL}(a, \mathbf{b}) = (ab_i)_{i=0}^{L-1}.$$

Operationally, the vector multiplier is evaluated by the following recurrence:

$$\begin{cases} \mathbf{z}_8 = \text{VSPLAT}(0), \\ \mathbf{b}_t = \text{VAND}(\text{VSHR}_t(\mathbf{b}), \text{VSPLAT}(1)), & t = 7, \dots, 0, \\ \mathbf{z}_t = \text{VXOR}(\text{GFXTIME}(\mathbf{z}_{t+1}), \text{VAND}(\text{VSPLAT}(a), \text{VSUB}(\text{VSPLAT}(0), \mathbf{b}_t))), & t = 7, \dots, 0. \end{cases}$$

Then

$$\text{GFVECMUL}(a, \mathbf{b}) = \mathbf{z}_0.$$

This definition is used for the Reed-Solomon syndrome computation and the vector Chien evaluation.

For data movement, VROT_s denotes a rotation of the physical vector by s bytes, and

$$\text{VDEAL2}(\mathbf{x}, \mathbf{y}, r) = (\mathbf{d}_{lo}, \mathbf{d}_{hi})$$

denotes a two-vector deinterleaving with grouping parameter r . The single-vector operation VDEALH denotes the corresponding halfword-lane deinterleaving. These permutation primitives are used to convert the scalar even/odd accesses in the RM Hadamard transform into vector-register operations.

We also use two reductions:

$$\text{VREDUCE_MAX}(\mathbf{x}) = \max_{0 \leq i < L} x_i, \quad \text{VREDUCE_MIN}(\mathbf{x}) = \min_{0 \leq i < L} x_i.$$

Both are expressed as rotation trees, e.g.,

$$\mathbf{z} \leftarrow \text{VMAX}(\mathbf{z}, \text{VROT}_s(\mathbf{z}))$$

for a sequence of rotations s . Replacing VMAX with VMIN gives the corresponding minimum reduction.

Finally, the RM transform uses the vector butterfly

$$\text{VBFLY}(\mathbf{x}, \mathbf{y}) = (\text{VADD}(\mathbf{x}, \mathbf{y}), \text{VSUB}(\mathbf{x}, \mathbf{y})),$$

and the selection logic uses

$$\begin{aligned} \text{VCMPEQ}(\mathbf{x}, \mathbf{y}) &= (\mathbf{1}\{x_i = y_i\})_{i=0}^{L-1}, \\ \text{VSELECT}(\mathbf{m}, \mathbf{x}, \mathbf{y}) &= (m_i x_i + (1 - m_i) y_i)_{i=0}^{L-1}. \end{aligned}$$

This mathematical model is the notation used in the following sections. The optimized implementations instantiate these operations with the available HVX permutation, arithmetic, mask, and reduction instructions, but the algorithmic description is independent of a particular intrinsic syntax.

Table 2. Mapping between the NPU abstraction and representative Qualcomm Hexagon HVX intrinsics.

Abstraction	HVX intrinsic realization	Role in the decoder
VADD	Q6_Vh_vadd_VhVh	Lane-wise halfword addition for RM butterflies.
VSUB	Q6_Vh_vsub_VhVh	Lane-wise halfword subtraction for RM butterflies and mask generation.
VABS	Q6_Vh_vabs_Vh	Absolute values of RM transform coefficients.
VMAX	Q6_Vh_vmax_VhVh	Lane-wise maximum in RM peak reduction.
VMIN	Q6_Vh_vmin_VhVh	Lane-wise minimum for the RM smallest-index tie break.
VSPLAT	Q6_Vh_vsplat_R	Broadcast scalar constants or field elements into all lanes.
VXOR	Q6_V_vxor_VV	Field addition and vector accumulator updates over $\mathbb{F}_{2^8}$.
VAND	Q6_V_vand_VV	Masking in vector finite-field multiplication and byte reduction.
VSHL _s	Q6_Vh_vasl_VhR	Lane-wise left shift in vector multiplication by X .
VSHR _s	Q6_Vuh_vlsr_VuhR	Lane-wise right shift for extracting multiplier and carry bits.
VROT _s	Q6_V_vror_VR	Rotation step in vector maximum and minimum reductions.
VDEAL2	Q6_W_vdeal_VVR	Two-vector deinterleaving for RM even/odd Hadamard inputs.
VDEALH	Q6_Vh_vdeal_Vh	Halfword-lane deinterleaving after the two-vector deal.
VCMPEQ	Q6_Q_vcmp_eq_VhVh	Predicate generation for RM peak-index selection.
VSELECT	Q6_V_vmux_QVV	Masked selection between valid indices and sentinel values.
VBFLY	Q6_Vh_vadd_VhVh and Q6_Vh_vsub_VhVh	Vector butterfly $(\mathbf{x} + \mathbf{y}, \mathbf{x} - \mathbf{y})$.
VREDUCE_MAX	Q6_Vh_vmax_VhVh with Q6_V_vror_VR	Tree reduction for the maximum RM correlation magnitude.
VREDUCE_MIN	Q6_Vh_vmin_VhVh with Q6_V_vror_VR	Tree reduction for the minimum index among tied RM peaks.
GFXTIME	VSHL ₁ , VSHR ₇ , VAND, VXOR, VSPLAT	Composite vector multiplication by X in $\mathbb{F}_{2^8}$.
GFVECMUL	Repeated GFXTIME, VAND, and VXOR	Composite scalar-by-vector finite-field multiplication for RS syndromes and Chien evaluation.

3 Proposed algorithms

Our HQC decoder implementation follows a two-stage concatenated decoding pipeline. The input word is first reorganized into Reed-Muller blocks, where each Reed-Solomon symbol corresponds to several duplicated Reed-Muller codewords. The Reed-Muller decoder processes these blocks independently through four main steps: expansion and accumulation, Hadamard transformation, bias correction, and peak detection. This stage converts the noisy binary input into a sequence of byte-level symbols, which becomes the input to the Reed-Solomon decoder. The Reed-Solomon decoder then performs syndrome computation, error-locator polynomial construction, error-location search, error-value computation, and final correction to recover the message. In our NPU-tailored implementation, each component is optimized according to its computational pattern: the Reed-Muller stage mainly benefits from SIMD-style element-wise arithmetic, vector reductions, and Hadamard butterfly operations, while the Reed-Solomon stage benefits from vectorized finite-field arithmetic, table lookup, comparison, and correction operations. This section mainly focuses on describing the optimization for 128-bit security version.

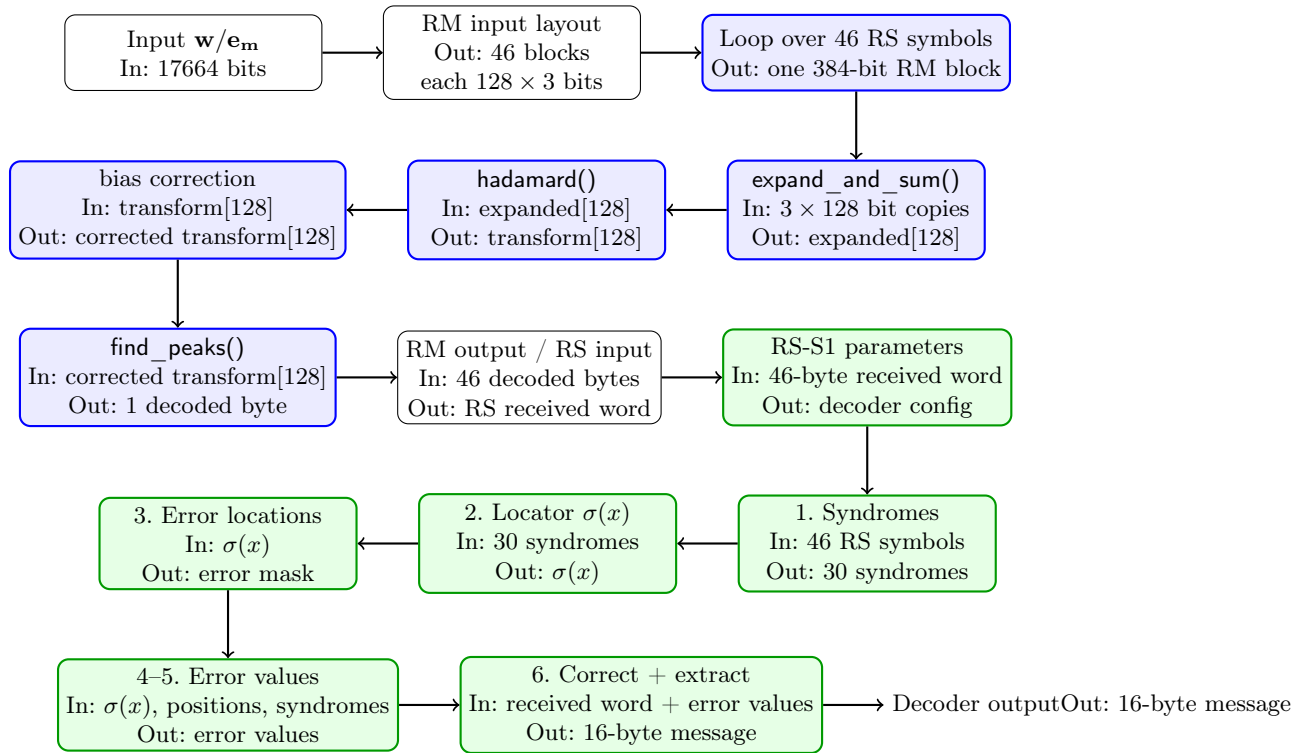


Fig. 1. Left-to-right block flow of the HQC-128 decoder. The flow wraps to a new row when the horizontal space is exceeded.

3.1 RM decoding

For each Reed-Solomon symbol, HQC-128 decodes one repeated RM(1,7) word of length 128 [3]. Since $n_2 = 384 = 3 \cdot 128$, the received RM block contains three repetitions. Let

$$b_i^{(r)} \in \{0, 1\}, \quad 0 \leq i < 128, \quad 0 \leq r < 3,$$

denote bit i in repetition r . The decoder first aggregates the integer-valued reliability vector

$$x_i = \sum_{r=0}^2 b_i^{(r)}, \quad x_i \in \{0, 1, 2, 3\}.$$

It then applies the RM(1, 7) fast Hadamard transform $\mathbf{H}_{128} = \mathbf{H}_2^{\otimes 7}$, where

$$\mathbf{H}_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Here, the $\{0, 1\}$ representation is used instead of the $\{-1, +1\}$ one, the first transform coefficient is corrected by $\hat{x}_0 \leftarrow \hat{x}_0 - 64 \cdot 3$. The decoded RM symbol is determined from the index

$$i^* = \min \arg \max_{0 \leq i < 128} |\hat{x}_i|,$$

with the sign of $\hat{x}_{i^*}$ encoded as the most significant output bit. The scalar reference computes these three stages sequentially: expansion, Hadamard transformation, and peak selection.

Packed repetition expansion The classical expansion step converts the three repeated RM words into one reliability vector. For each coordinate i , the decoder counts how many repetitions contain a one:

$$x_i = b_i^{(0)} + b_i^{(1)} + b_i^{(2)}.$$

This produces the integer input $\mathbf{x} \in \{0, 1, 2, 3\}^{128}$ for the Hadamard decoder.

The NPU-oriented expansion is the same map, but grouped by nibbles. Let $n_{r,p,q} \in \{0, \dots, 15\}$ be nibble q of 32-bit part p in repetition r . Define

$$E(n) = (n_0, n_1, n_2, n_3) \in \{0, 1\}^4,$$

where n_ℓ is bit ℓ of n . For each part and nibble, the optimized decoder forms

$$\mathbf{e}_{p,q} = E(n_{0,p,q}) + E(n_{1,p,q}) + E(n_{2,p,q}) \in \{0, 1, 2, 3\}^4.$$

Concatenating the $4 \cdot 8$ vectors $\mathbf{e}_{p,q}$ yields exactly the same vector $\mathbf{x} = (x_0, \dots, x_{127})$ as the scalar expansion. The optimized expansion schedule is therefore

$$(p, q) \mapsto \mathbf{e}_{p,q} = E(n_{0,p,q}) + E(n_{1,p,q}) + E(n_{2,p,q}), \quad 0 \leq p < 4, 0 \leq q < 8.$$

Vector Hadamard transformation After expansion, RM decoding applies the fast Hadamard transform for RM(1, 7). The transform is computed by seven butterfly layers. Each layer applies 64 independent butterflies

$$(u, v) \mapsto (u + v, u - v).$$

The NPU-oriented schedule replaces the scalar even/odd gather with vector permutations. Let

$$\mathbf{lo} = (x_0, \dots, x_{63}), \quad \mathbf{hi} = (x_{64}, \dots, x_{127}).$$

One pass is expressed as

$$\begin{aligned} (\mathbf{d}_{lo}, \mathbf{d}_{hi}) &= \text{VDEAL2}(\mathbf{hi}, \mathbf{lo}, 2), \\ \mathbf{v}_e &= \text{VDEALH}(\mathbf{d}_{lo}), \quad \mathbf{v}_o = \text{VDEALH}(\mathbf{d}_{hi}), \end{aligned}$$

where $\mathbf{v}_e = (x_0, x_2, \dots, x_{126})$ and $\mathbf{v}_o = (x_1, x_3, \dots, x_{127})$ after the deinterleaving. The butterflies are then computed as

$$(\mathbf{y}_{0:63}, \mathbf{y}_{64:127}) = \text{VBFLY}(\mathbf{v}_e, \mathbf{v}_o).$$

For pass $t = 0, \dots, 6$, the algorithmic schedule is

1. $(\mathbf{d}_{lo}, \mathbf{d}_{hi}) \leftarrow \text{VDEAL2}(\mathbf{hi}, \mathbf{lo}, 2)$,
2. $\mathbf{v}_e \leftarrow \text{VDEALH}(\mathbf{d}_{lo})$, $\mathbf{v}_o \leftarrow \text{VDEALH}(\mathbf{d}_{hi})$,
3. $(\mathbf{y}_{0:63}, \mathbf{y}_{64:127}) \leftarrow \text{VBFLY}(\mathbf{v}_e, \mathbf{v}_o)$.

The 64 butterflies in step 3 are evaluated in parallel. The pass index t remains sequential because pass $t + 1$ consumes the output of pass t .

Vector peak selection with scalar-equivalent tie breaking The final RM decision is maximum-correlation decoding. Given the Hadamard output $\hat{\mathbf{x}}$, the decoder chooses

$$i^* = \min \arg \max_i |\hat{x}_i|.$$

The minimum is part of the algorithm: if two coefficients have equal magnitude, the smaller index is selected. The sign of $\hat{x}_{i^*}$ determines the most significant output bit.

Let

$$\begin{cases} \mathbf{a}_0 = \text{VABS}(\mathbf{r}_0), \\ \mathbf{a}_1 = \text{VABS}(\mathbf{r}_1), \\ A^* = \text{VREDUCE_MAX}(\text{VMAX}(\mathbf{a}_0, \mathbf{a}_1)) \\ \quad = \max_{0 \leq i < 128} |\hat{x}_i|. \end{cases}$$

Let $\mathbf{i}_0 = (0, \dots, 63)$, $\mathbf{i}_1 = (64, \dots, 127)$, and let ∞ denote a sentinel larger than any valid RM index. Candidate indices are selected by

$$\begin{aligned} \mathbf{p}_0 &= \text{VSELECT}(\text{VCMPEQ}(\mathbf{a}_0, \text{VSPLAT}(A^*)), \mathbf{i}_0, \text{VSPLAT}(\infty)), \\ \mathbf{p}_1 &= \text{VSELECT}(\text{VCMPEQ}(\mathbf{a}_1, \text{VSPLAT}(A^*)), \mathbf{i}_1, \text{VSPLAT}(\infty)). \end{aligned}$$

The final position is $i^* = \text{VREDUCE_MIN}(\text{VMIN}(\mathbf{p}_0, \mathbf{p}_1))$.

Therefore, the peak-selection schedule is

1. $\mathbf{a}_0, \mathbf{a}_1 \leftarrow \text{VABS}(\mathbf{r}_0), \text{VABS}(\mathbf{r}_1)$,
2. $A^* \leftarrow \text{VREDUCE_MAX}(\text{VMAX}(\mathbf{a}_0, \mathbf{a}_1))$,
3. $\mathbf{p}_0, \mathbf{p}_1 \leftarrow$ masked index vectors for lanes equal to A^* ,
4. $i^* \leftarrow \text{VREDUCE_MIN}(\text{VMIN}(\mathbf{p}_0, \mathbf{p}_1))$.

Steps 1 and 3 are lane-parallel over coefficients; steps 2 and 4 are reduction trees. The final sign bit is read from the selected coefficient $\hat{x}_{i^*}$.

3.2 RS decoding

For HQC-128, the shortened Reed-Solomon code has length $n_1 = 46$, message length $k = 16$, and error-correction parameter $\delta = 15$. All Reed-Solomon algebra is over $\mathbb{F}_{2^8}$, represented with the primitive polynomial

$$P(X) = X^8 + X^4 + X^3 + X^2 + 1.$$

Given a received word

$$\mathbf{c} = (c_0, \dots, c_{45}),$$

the decoder computes $2\delta = 30$ syndromes, derives the error-locator polynomial $\sigma(x)$, identifies its roots over the shortened support, computes error magnitudes, and corrects the received word [?].

The HQC specification describes the root-finding step using an additive FFT. In our modified implementation, the reference FFT backend is retained for differential testing, while the default shortened-support backend evaluates the locator polynomial directly on the public RS support. The HVX root-search backend further vectorizes this evaluation over all HQC-128 support positions.

Vectorized syndrome evaluation Syndrome computation evaluates the received word at consecutive nonzero powers of the primitive element. Mathematically, the Reed-Solomon syndromes are

$$S_i = c_0 \oplus \bigoplus_{j=1}^{n_1-1} c_j \alpha^{ij}, \quad 1 \leq i \leq 2\delta.$$

The implementation stores these values in a zero-indexed array. Thus,

$$\tilde{S}_i = S_{i+1}, \quad 0 \leq i < 2\delta,$$

and therefore

$$\tilde{S}_i = c_0 \oplus \bigoplus_{j=1}^{n_1-1} c_j \alpha^{(i+1)j}, \quad 0 \leq i < 2\delta.$$

The scalar algorithm fixes i , accumulates all received-symbol contributions over j , and repeats this for all 2δ syndromes. To expose vector parallelism, we transpose the power table and define, for each received-symbol index j ,

$$\mathbf{a}_j = (\alpha^j, \alpha^{2j}, \dots, \alpha^{(2\delta)j}, 0, \dots, 0) \in \mathbb{F}_{2^8}^L, \quad 1 \leq j < n_1,$$

where the remaining lanes are padding. Let

$$\text{GFVECMUL}(a, \mathbf{b}) = (ab_\ell)_{\ell=0}^{L-1}$$

denote lane-wise multiplication in $\mathbb{F}_{2^8}$. The vector syndrome recurrence is

$$\begin{cases} \mathbf{s}^{(0)} = \text{VSPLAT}(0), \\ \mathbf{s}^{(j)} = \text{VXOR}(\mathbf{s}^{(j-1)}, \text{GFVECMUL}(c_j, \mathbf{a}_j)), \end{cases} \quad 1 \leq j < n_1.$$

After all nonconstant powers have been accumulated, the c_0 contribution is added by

$$\mathbf{S} = \text{VXOR}(\mathbf{s}^{(n_1-1)}, \text{VSPLAT}(c_0)).$$

The first 2δ lanes of $\mathbf{S}$ are the required syndrome array

$$(\tilde{S}_0, \dots, \tilde{S}_{2\delta-1}).$$

The schedule is

1. $\mathbf{s} \leftarrow \text{VSPLAT}(0)$,
2. for $j = 1, \dots, n_1 - 1$: $\mathbf{s} \leftarrow \text{VXOR}(\mathbf{s}, \text{GFVECMUL}(c_j, \mathbf{a}_j))$,
3. $\mathbf{S} \leftarrow \text{VXOR}(\mathbf{s}, \text{VSPLAT}(c_0))$.

For fixed j , step 2 evaluates all products

$$(c_j \alpha^j, c_j \alpha^{2j}, \dots, c_j \alpha^{(2\delta)j})$$

in parallel. The loop over j remains sequential because all received symbols contribute to the same syndrome vector.

Vector finite-field multiplication Reed-Solomon decoding repeatedly multiplies field elements in $\mathbb{F}_{2^8}$. The scalar baseline can be viewed as a shift-and-add multiplication: scan the multiplier bits, repeatedly multiply the accumulator by X , and conditionally add the multiplicand. Reduction is performed modulo

$$P(X) = X^8 + X^4 + X^3 + X^2 + 1.$$

The NPU-oriented product GFVECMUL applies the same algorithm to many multipliers in parallel. For a vector $\mathbf{x}$ whose lanes contain field elements, multiplication by X modulo $P(X)$ is

$$\text{GFXTIME}(\mathbf{x}) = \text{VAND}\left(\text{VXOR}(\text{VSHL}_1(\mathbf{x}), \text{VAND}(\mathbf{m}, \text{VSPLAT}(0\mathbf{x}1\mathbf{d}))), \text{VSPLAT}(0\mathbf{x}\mathbf{f}\mathbf{f})\right),$$

where

$$\mathbf{m} = \text{VSUB}(\text{VSPLAT}(0), \text{VAND}(\text{VSHR}_7(\mathbf{x}), \text{VSPLAT}(1))).$$

For a scalar multiplicand a and a vector multiplier $\mathbf{b}$, the lane-wise product is evaluated by

$$\begin{cases} \mathbf{z}_8 = \text{VSPLAT}(0), \\ \mathbf{b}_t = \text{VAND}(\text{VSHR}_t(\mathbf{b}), \text{VSPLAT}(1)), & t = 7, \dots, 0, \\ \mathbf{z}_t = \text{VXOR}\left(\text{GFXTIME}(\mathbf{z}_{t+1}), \text{VAND}(\text{VSPLAT}(a), \text{VSUB}(\text{VSPLAT}(0), \mathbf{b}_t))\right), & t = 7, \dots, 0. \end{cases}$$

The output is $\text{GFVECMUL}(a, \mathbf{b}) = \mathbf{z}_0$.
Algorithmically,

1. $\mathbf{z}_8 \leftarrow \text{VSPLAT}(0)$,
2. for $t = 7, \dots, 0$: $\mathbf{z}_t \leftarrow \text{GFXTIME}(\mathbf{z}_{t+1}) \oplus (a \text{ masked by bit } t \text{ of } \mathbf{b})$,
3. $\text{GFVECMUL}(a, \mathbf{b}) \leftarrow \mathbf{z}_0$.

The loop over t is sequential, but each step is parallel over all lanes. Besides, we also provide look-up-table-based multiplication style.

Error-locator polynomial The error-locator polynomial identifies the error positions. If the error location numbers are

$$\beta_1, \dots, \beta_t,$$

then

$$\sigma(x) = \prod_{h=1}^t (1 + \beta_h x) = 1 + \sigma_1 x + \dots + \sigma_t x^t.$$

The implementation computes $\sigma(x)$ using the Berlekamp-Massey recurrence [1]. Using the zero-indexed syndrome array $\tilde{S}_\mu = S_{\mu+1}$, the discrepancy at iteration μ is

$$d_\mu = \tilde{S}_\mu \oplus \bigoplus_{i=1}^{\deg \sigma} \sigma_i \tilde{S}_{\mu-i}.$$

If $d_\mu = 0$, the current locator polynomial already explains the syndrome sequence up to that iteration. If $d_\mu \neq 0$, the polynomial is corrected using an auxiliary polynomial $b(x)$, a shift m , and the previous nonzero discrepancy d_p . The update factor is

$$\gamma = d_\mu d_p^{-1},$$

and the polynomial update is

$$\sigma(x) \leftarrow \sigma(x) \oplus \gamma x^m b(x).$$

Coefficient-wise, this is

$$\sigma_i \leftarrow \sigma_i \oplus \gamma b_{i-m}.$$

In the benchmark fast path, we explicitly track $\deg \sigma$, $\deg b$, and m . This allows the update loop to touch only coefficients that can be nonzero:

$$m \leq i \leq \min(\delta, m + \deg b).$$

This is an algebraic work reduction rather than a lane-parallel stage. The fast path can be summarized as

1. for $\mu = 0, \dots, 2\delta - 1$ compute d_μ ,
2. if $d_\mu \neq 0$, set $\gamma = d_\mu d_p^{-1}$ and update only $i \in [m, \min(\delta, m + \deg b)]$,
3. if $2 \deg \sigma \leq \mu$, update $\deg \sigma$, $b(x)$, $\deg b$, m , and d_p ,
4. otherwise increment m .

The loop over μ remains sequential because each iteration depends on the current $\sigma(x)$. implementation.

Shortened-support root search Root finding evaluates the locator polynomial on the code support. For a candidate position i , define $\beta_i = \alpha^i, x_i = \beta_i^{-1} = \alpha^{-i}, 0 \leq i < n_1$. A position is erroneous exactly when $\sigma(x_i) = 0$. The HQC specification describes this root-finding step using an additive FFT. In the modified implementation, the default shortened-support backend evaluates $\sigma(x_i)$ only on the n_1 public support points. For HQC-128, this means only $n_1 = 46$ candidate positions are tested.

The NPU-oriented Chien schedule precomputes

$$\mathbf{p}_j = (x_0^j, x_1^j, \dots, x_{n_1-1}^j, 0, \dots, 0) \in \mathbb{F}_{2^8}^L, \quad 0 \leq j \leq \delta.$$

It then evaluates all support points as

$$\mathbf{r} = \bigoplus_{j=0}^{\deg \sigma} \text{GFVECMUL}(\sigma_j, \mathbf{p}_j).$$

The located-error indicator is $e_i^{\text{loc}} = \mathbf{1}\{r_i = 0\}, 0 \leq i < n_1$.

This is a direct vector form of Chien search [2]. The schedule is

1. $\mathbf{r} \leftarrow \text{VSPLAT}(0)$,
2. for $j = 0, \dots, \deg \sigma$: $\mathbf{r} \leftarrow \text{VXOR}(\mathbf{r}, \text{GFVECMUL}(\sigma_j, \mathbf{p}_j))$,
3. $e_i^{\text{loc}} \leftarrow \mathbf{1}\{r_i = 0\}, \quad 0 \leq i < n_1$.

For fixed j , all support points $x_i = \alpha^{-i}$ are evaluated in parallel. The loop over j remains sequential because it accumulates the polynomial value.

This schedule is particularly suitable for HQC-128 because the shortened support has only 46 valid positions and therefore fits into a single HVX vector with padding lanes. The padding lanes are ignored when writing the error indicator array.

Error evaluator and Forney denominator After the error locations are known, the decoder computes the corresponding error magnitudes. Let

$$z(x) = 1 + z_1x + \dots + z_\delta x^\delta$$

be the error-evaluator polynomial. In the implementation, $z(x)$ is computed from $\sigma(x)$ and the syndrome sequence. In the benchmark fast path, only coefficients up to the actual locator degree are computed.

For a located error at position i , let $\beta_i = \alpha^i, x_i = \beta_i^{-1}$. The numerator is

$$N_i = z(x_i) = 1 \oplus \bigoplus_{j=1}^{\deg \sigma} z_j \beta_i^{-j}.$$

The scalar reference computes the denominator as a product over the other located positions:

$$\prod_{h \neq i} (1 + \beta_h \beta_i^{-1}).$$

The optimized benchmark path replaces this product with the derivative form of Forney's formula. Since

$$\sigma(x) = \prod_{h=1}^t (1 + \beta_h x),$$

we have, at the root $x_i = \beta_i^{-1}$,

$$\sigma'(\beta_i^{-1}) = \beta_i \prod_{h \neq i} (1 + \beta_h \beta_i^{-1}),$$

$$\prod_{h \neq i} (1 + \beta_h \beta_i^{-1}) = \beta_i^{-1} \sigma'(\beta_i^{-1}).$$

Thus the error value can be computed as

$$E_i = \frac{N_i}{\prod_{h \neq i} (1 + \beta_h \beta_i^{-1})} = N_i \beta_i (\sigma'(\beta_i^{-1}))^{-1}.$$

In characteristic two, the formal derivative keeps only the odd-degree coefficients:

$$\sigma'(x) = \sum_{\substack{1 \leq j \leq \deg \sigma \\ j \text{ odd}}} \sigma_j x^{j-1}.$$

Therefore, for each located error, the optimized path evaluates

$$\begin{cases} N_i = 1 \oplus \bigoplus_{j=1}^{\deg \sigma} z_j \beta_i^{-j}, \\ D_i = \sigma'(\beta_i^{-1}) = \bigoplus_{\substack{1 \leq j \leq \deg \sigma \\ j \text{ odd}}} \sigma_j \beta_i^{-(j-1)}, \\ E_i = N_i \beta_i D_i^{-1}. \end{cases}$$

The optimized error-value schedule is

1. $\mathcal{E} \leftarrow \{i : e_i^{\text{loc}} = 1\}$,
2. for $i \in \mathcal{E}$ compute $N_i = 1 \oplus \bigoplus_{j=1}^{\deg \sigma} z_j \beta_i^{-j}$,
3. compute $D_i = \bigoplus_{\substack{1 \leq j \leq \deg \sigma \\ j \text{ odd}}} \sigma_j \beta_i^{-(j-1)}$,
4. $E_i \leftarrow N_i \beta_i D_i^{-1}$.

Compared with the fixed-bound scalar product formulation, this benchmark path has two advantages in the performance setting. First, the loop bound is the actual locator degree rather than the fixed correction capacity δ . Second, only odd locator coefficients participate in the denominator because even-degree terms vanish in the formal derivative.

4 Benchmark

4.1 Benchmark Methodology

The corpus contains 16 deterministic random codewords whose Reed–Solomon symbol-error counts range from 0 to 15. The benchmark binary contains only the decoder and its dependencies, so the reported cost isolates the HQC decapsulation recovery path rather than key generation or encryption.

The Hexagon simulator reports total processor cycles for complete corpus runs. To remove fixed simulator startup and boot overhead, we estimate the per-decode cost from the difference between a 10-iteration corpus run and a 1-iteration corpus run:

$$C_{\text{decode}} = \frac{C_{10} - C_1}{9 \cdot 16},$$

where C_{10} and C_1 denote the corresponding total simulator Pcycles. All tables below use this estimator. We report Pcycles rather than wall-clock time because simulator wall-clock time includes host scheduling and simulator startup effects.

We distinguish two benchmark classes. The conservative intrinsic path preserves the fixed-flow Reed–Solomon arithmetic used by the scalar reference and is the main constant-flow comparison point. The side-channel-relaxed endpoints combine branchy Reed–Solomon algebra and lookup-table multiplication; they are included to measure the performance ceiling of the NPU reformulation, not as a production security recommendation.

4.2 Full-Decoder Results

Table 3 summarizes the full HQC-128 decoder cost as successive algorithmic components are moved from the scalar formulation to the NPU-oriented formulation. The scalar baseline is the unvectorized implementation in `hqc_lab_scalar`. The intrinsic rows correspond to the optimized implementation in `hqc_lab_intrinsic`.

The conservative intrinsic decoder reduces the estimated cost from 953,767 to 396,505 Pcycles per decode, a 58.4% reduction relative to the scalar reference. This is the most direct measurement of the NPU formulation studied in this paper because it keeps the decoder algebraic flow close to the scalar implementation. The fastest endpoint reaches 58,602 Pcycles per decode, corresponding to a 93.9% reduction, after also enabling lookup-table field arithmetic, table-assisted RM expansion, fused RM decoding, tighter RS recurrences, and vector Chien root search.

Table 3. Full HQC-128 decode cost on the 16-fixture corpus.

Configuration	Optimization represented	Est. Pcycles/decode	Reduction
Scalar reference	Sequential RS and RM decoding	953,767	0.0%
HVX syndrome endpoint	First RS syndrome vectorization	478,072	49.9%
Conservative intrinsic path	Fixed-flow RS arithmetic with vector RM stages	396,505	58.4%
Fast RS algebra with GF lookup	Side-channel-relaxed RS algebra and GF multiplication tables	149,482	84.3%
GF table backend	Table-backed field multiplication throughout the fast path	111,361	88.3%
GF table plus RM expansion table	Lookup-based RM expansion added to the fast RS path	67,034	93.0%
Fused RM endpoint	RM expansion, transform, and decision fused more tightly	66,620	93.0%
Tighter RS algebra	Degree-bounded RS recurrences and reduced scalar overhead	60,834	93.6%
HVX root-search endpoint	Vector Chien evaluation added to the fastest path	58,602	93.9%

4.3 Substage Profile

Table 4 gives the measured substage costs for the default intrinsic path. These measurements are separate harness measurements, so they should be read as a profile of where the optimized decoder spends time rather than as an exact additive decomposition of Table 3.

Table 4. Default intrinsic substage profile.

Substage	Algorithmic role	Est. Pcycles/decode	Optimization implication
RM expansion and summation	Repetition aggregation into RM reliabilities	55,902	Still visible because byte unpacking and signed accumulation dominate before the Hadamard transform.
RM Hadamard transform	Seven-layer Walsh–Hadamard correlation	18,907	No longer a dominant cost after vector butterflies replace scalar gathers.
RM peak selection	Maximum absolute correlation and sign recovery	5,666	Vector reductions make the final RM decision comparatively small.
RS syndrome computation	Evaluation of the received word at consecutive field powers	3,726	The transposed vector schedule removes this stage as a bottleneck.
RS error-locator polynomial	Berlekamp–Massey recurrence over bounded degree	109,691	Remains costly because each discrepancy update carries algebraic dependencies.
RS root search	Locator evaluation over the shortened support	75,246	Benefits from vector Chien evaluation in the fastest endpoint.
RS $z(x)$ polynomial	Error-evaluator construction	11,784	Smaller than locator and value computation after degree bounds are applied.
RS error values	Forney numerator and denominator evaluation	131,046	Largest remaining default-path cost due to multiple finite-field products per located error.
RS correction	Application of recovered symbol errors	440	Negligible after error locations and values have been computed.

The profile explains why the later endpoints concentrate on RS algebra and lookup-table field multiplication rather than on further Hadamard or syndrome optimization. In the conservative intrinsic path, the

Hadamard transform, RM peak search, and syndrome computation have already been reduced to small components of the total decoder cost. The remaining performance pressure is in finite-field-heavy RS work: error values, the error-locator polynomial, and root evaluation.

4.4 Benchmark Summary

Table 5. Selected milestones relative to the scalar implementation.

Milestone	Est. Pcycles/decode	Reduction Interpretation
Scalar reference	953,767	0.0% Baseline sequential RS and RM decoder.
Conservative intrinsic path	396,505	58.4% Main fixed-flow NPU comparison point.
Fast RS algebra with GF lookup	149,482	84.3% Shows the effect of relaxing the RS arithmetic path.
GF table plus RM expansion table	67,034	93.0% Shows the combined benefit of table-assisted GF and RM stages.
Final fastest endpoint	58,602	93.9% Measures the observed performance ceiling for the optimized decoder.

Overall, the measurements support the algorithmic decomposition used in Sections ?? and the proposed RS/RM methods. The NPU abstraction gives the largest conservative speedup when independent RM coefficients, Hadamard butterflies, and syndrome lanes are evaluated in parallel. Once those stages are vectorized, the dominant residual work is field multiplication inside the Reed–Solomon decoder, which is why the fastest benchmark endpoints are driven by GF multiplication tables, degree-bounded RS recurrences, and vectorized root search.

References

1. E. R. Berlekamp. *Algebraic Coding Theory*. McGraw-Hill, 1968.
2. R. T. Chien. Cyclic decoding procedures for Bose-Chaudhuri-Hocquenghem codes. *IEEE Transactions on Information Theory*, 10(4):357–363, 1964.
3. F. J. MacWilliams and N. J. A. Sloane. *The Theory of Error-Correcting Codes*. North-Holland, 1977.
4. National Institute of Standards and Technology. Module-lattice-based digital signature standard. Federal Information Processing Standards Publication 204, National Institute of Standards and Technology, 2024.
5. National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication 203, National Institute of Standards and Technology, 2024.
6. National Institute of Standards and Technology. Stateless hash-based digital signature standard. Federal Information Processing Standards Publication 205, National Institute of Standards and Technology, 2024.
7. National Institute of Standards and Technology. Status report on the fourth round of the NIST post-quantum cryptography standardization process. NIST Interagency/Internal Report 8545, National Institute of Standards and Technology, 2025.
8. Qualcomm Technologies, Inc. *Qualcomm Hexagon V73 HVX Programmer’s Reference Manual*, 2024. 80-N2040-54 Rev. AB, January 29, 2024.
9. Qualcomm Technologies, Inc. *Qualcomm Hexagon V79 HVX Programmer Reference Manual*, 2025. 80-N2040-61, January 16, 2025.
10. Qualcomm Technologies, Inc. *Qualcomm Hexagon V81 HMX Programmer’s Reference Manual*, 2026. 80-N2040-62 Rev. AA, March 2026.

5 Appendix

5.1 Pseudocode

Algorithm 1 HVX-based Hadamard transform

Require: Input buffer `src` containing 128 signed 16-bit values**Require:** Temporary buffer `dst` containing 128 signed 16-bit values**Ensure:** Hadamard-transformed values are stored in the buffer pointed to by p_1

```

1:  $p_1 \leftarrow \text{src}$ 
2:  $p_2 \leftarrow \text{dst}$ 
3: for  $\text{pass} \leftarrow 0$  to 6 do
4:    $\text{lo} \leftarrow p_1[0 : 63]$ 
5:    $\text{hi} \leftarrow p_1[64 : 127]$ 
6:    $(\mathbf{d}_{lo}, \mathbf{d}_{hi}) \leftarrow \text{VDEAL}(\text{hi}, \text{lo}, 2)$ 
7:    $\mathbf{v}_e \leftarrow \text{VDEALH}(\mathbf{d}_{lo})$ 
8:    $\mathbf{v}_o \leftarrow \text{VDEALH}(\mathbf{d}_{hi})$ 
9:    $\text{sum} \leftarrow \text{VADDH}(\mathbf{v}_e, \mathbf{v}_o)$ 
10:   $\text{diff} \leftarrow \text{VSUBH}(\mathbf{v}_e, \mathbf{v}_o)$ 
11:   $p_2[0 : 63] \leftarrow \text{sum}$ 
12:   $p_2[64 : 127] \leftarrow \text{diff}$ 
13:   $\text{swap}(p_1, p_2)$ 
14: end for
15: return  $p_1$ 

```

Algorithm 2 HVX-based peak detection

Require: Hadamard-transformed buffer `transform` containing 128 signed 16-bit values.**Ensure:** Encoded peak position used for Reed-Muller decoding.

```

1:  $\text{row}_0 \leftarrow \text{transform}[0 : 63]$ 
2:  $\text{row}_1 \leftarrow \text{transform}[64 : 127]$ 
3:  $\text{abs}_0 \leftarrow \text{VABS}(\text{row}_0)$ 
4:  $\text{abs}_1 \leftarrow \text{VABS}(\text{row}_1)$ 
5:  $\text{max} \leftarrow \text{VMAX}(\text{abs}_0, \text{abs}_1)$ 
6: for  $s \in \{64, 32, 16, 8, 4, 2\}$  do
7:    $\text{max} \leftarrow \text{VMAX}(\text{max}, \text{VROT}_s(\text{max}))$ 
8: end for
9:  $a^* \leftarrow \text{max}[0]$ 
10:  $\text{target} \leftarrow \text{VSPLAT}(a^*)$ 
11:  $\text{sentinel} \leftarrow \text{VSPLAT}(0x7fff)$ 
12:  $\text{idx}_0 \leftarrow \text{rm\_index\_lo}$ 
13:  $\text{idx}_1 \leftarrow \text{rm\_index\_hi}$ 
14:  $\text{mask}_0 \leftarrow \text{VCMPEQ}(\text{abs}_0, \text{target})$ 
15:  $\text{mask}_1 \leftarrow \text{VCMPEQ}(\text{abs}_1, \text{target})$ 
16:  $\text{pos}_0 \leftarrow \text{VSELECT}(\text{mask}_0, \text{idx}_0, \text{sentinel})$ 
17:  $\text{pos}_1 \leftarrow \text{VSELECT}(\text{mask}_1, \text{idx}_1, \text{sentinel})$ 
18:  $\text{peak} \leftarrow \text{VMIN}(\text{pos}_0, \text{pos}_1)$ 
19: for  $s \in \{64, 32, 16, 8, 4, 2\}$  do
20:    $\text{peak} \leftarrow \text{VMIN}(\text{peak}, \text{VROT}_s(\text{peak}))$ 
21: end for
22:  $i^* \leftarrow \text{peak}[0]$ 
23:  $v^* \leftarrow \text{transform}[i^*]$ 
24: if  $v^* > 0$  then
25:    $i^* \leftarrow i^* \mid 128$ 
26: end if
27: return  $i^*$ 

```

Algorithm 3 HVX-optimized syndrome computation

Require: Received word $\mathbf{c} = (c_0, \dots, c_{n_1-1})$
Require: Transposed table alpha_ji_pow with $\text{alpha_ji_pow}[j][i] = \alpha^{ij}$
Ensure: Syndrome vector $\mathbf{S} = (S_0, \dots, S_{2\delta-1})$

```

1:  $\text{acc} \leftarrow \text{VZERO}()$ 
2: for  $j \leftarrow 1$  to  $n_1 - 1$  do
3:    $\mathbf{a} \leftarrow \text{alpha\_ji\_pow}[j - 1][0 : 63]$ 
4:    $\mathbf{p} \leftarrow \text{GF\_MUL\_SCALAR\_VEC}(c_j, \mathbf{a})$ 
5:    $\text{acc} \leftarrow \text{VXOR}(\text{acc}, \mathbf{p})$ 
6: end for
7:  $\text{acc} \leftarrow \text{VXOR}(\text{acc}, \text{VSPLAT}(c_0))$ 
8:  $\mathbf{S}[0 : 2\delta - 1] \leftarrow \text{acc}[0 : 2\delta - 1]$ 
9: return  $\mathbf{S}$ 

```

Algorithm 4 Vectorized multiplication by x over $\mathbb{F}_{2^8}$

Require: Vector $\mathbf{x}$ whose lanes contain elements of $\mathbb{F}_{2^8}$
Ensure: Vector $\mathbf{y}$ where $y_i = X \cdot x_i \bmod P(X)$

```

1:  $\text{carry} \leftarrow \text{VAND}(\text{VSHR}_7(\mathbf{x}), \text{VSPLAT}(1))$ 
2:  $\text{carry\_mask} \leftarrow \text{VSUB}(\text{VZERO}(), \text{carry})$ 
3:  $\text{shifted} \leftarrow \text{VSHL}_1(\mathbf{x})$ 
4:  $\text{reduction} \leftarrow \text{VAND}(\text{carry\_mask}, \text{VSPLAT}(0x1d))$ 
5:  $\mathbf{y} \leftarrow \text{VXOR}(\text{shifted}, \text{reduction})$ 
6:  $\mathbf{y} \leftarrow \text{VAND}(\mathbf{y}, \text{VSPLAT}(0xff))$ 
7: return  $\mathbf{y}$ 

```

Algorithm 5 Vectorized scalar-by-vector multiplication over $\mathbb{F}_{2^8}$

Require: Scalar field element $a \in \mathbb{F}_{2^8}$
Require: Vector $\mathbf{b} = (b_0, \dots, b_{L-1})$ of field elements
Ensure: Vector $(ab_0, \dots, ab_{L-1})$

```

1:  $\text{acc} \leftarrow \text{VZERO}()$ 
2:  $\mathbf{a} \leftarrow \text{VSPLAT}(a)$ 
3:  $\text{one} \leftarrow \text{VSPLAT}(1)$ 
4:  $\text{zero} \leftarrow \text{VZERO}()$ 
5: for  $t \leftarrow 7$  downto  $0$  do
6:    $\mathbf{b}_t \leftarrow \text{VAND}(\text{VSHR}_t(\mathbf{b}), \text{one})$ 
7:    $\text{mask} \leftarrow \text{VSUB}(\text{zero}, \mathbf{b}_t)$ 
8:    $\text{acc} \leftarrow \text{GF\_XTIME\_VEC}(\text{acc})$ 
9:    $\text{acc} \leftarrow \text{VXOR}(\text{acc}, \text{VAND}(\mathbf{a}, \text{mask}))$ 
10: end for
11: return  $\text{acc}$ 

```

Algorithm 6 Hardware-style multiplication over $\mathbb{F}_{2^8}$

Require: Field elements $a, b \in \mathbb{F}_{2^8}$
Ensure: Product ab

```

1:  $\text{acc} \leftarrow 0$ 
2:  $a \leftarrow a \& 0xff$ 
3:  $b \leftarrow b \& 0xff$ 
4: for  $t \leftarrow 7$  downto  $0$  do
5:    $\text{bit\_mask} \leftarrow -((b \gg t) \& 1)$ 
6:    $\text{acc} \leftarrow \text{XTIME}(\text{acc})$ 
7:    $\text{acc} \leftarrow \text{acc} \oplus (a \& \text{bit\_mask})$ 
8: end for
9: return  $\text{acc}$ 

```

Algorithm 7 Fast non-constant-time error-locator polynomial computation

Require: Syndrome vector $\mathbf{S} = (S_0, \dots, S_{2\delta-1})$
Ensure: Error-locator polynomial $\sigma(x)$ and degree d_σ

```

1:  $\sigma(x) \leftarrow 1$ 
2:  $b(x) \leftarrow 1$ 
3:  $d_\sigma \leftarrow 0$ 
4:  $m \leftarrow 1$ 
5:  $d_p \leftarrow 1$ 
6: for  $\mu \leftarrow 0$  to  $2\delta - 1$  do
7:    $d \leftarrow S_\mu$ 
8:   for  $i \leftarrow 1$  to  $d_\sigma$  do
9:      $d \leftarrow d \oplus \sigma_i S_{\mu-i}$ 
10:  end for
11:  if  $d \neq 0$  then
12:     $\gamma \leftarrow d \cdot d_p^{-1}$ 
13:     $t(x) \leftarrow \sigma(x)$ 
14:    for  $i \leftarrow m$  to  $\delta$  do
15:       $\sigma_i \leftarrow \sigma_i \oplus \gamma b_{i-m}$ 
16:    end for
17:    if  $2d_\sigma \leq \mu$  then
18:       $d_\sigma \leftarrow \mu + 1 - d_\sigma$ 
19:       $b(x) \leftarrow t(x)$ 
20:       $m \leftarrow 1$ 
21:       $d_p \leftarrow d$ 
22:    else
23:       $m \leftarrow m + 1$ 
24:    end if
25:  else
26:     $m \leftarrow m + 1$ 
27:  end if
28: end for
29: return  $(\sigma, d_\sigma)$ 

```

Algorithm 8 Fast non-constant-time error-value computation

Require: Error-evaluator polynomial $z(x)$
Require: Error-location mask $\mathbf{e}_{loc}$
Ensure: Error-value vector $\mathbf{e}_{val}$

```

1:  $\mathcal{E} \leftarrow \emptyset$ 
2: for  $i \leftarrow 0$  to  $n_1 - 1$  do
3:   if  $\mathbf{e}_{loc}[i] = 1$  and  $|\mathcal{E}| < \delta$  then
4:     Add  $i$  to  $\mathcal{E}$ 
5:   end if
6: end for
7: for all  $i \in \mathcal{E}$  do
8:    $\beta_i \leftarrow \alpha^i$ 
9:    $\beta_i^{-1} \leftarrow \text{GFInv}(\beta_i)$ 
10:   $\text{invpow} \leftarrow 1$ 
11:   $N \leftarrow 1$ 
12:   $D \leftarrow 1$ 
13:  for  $j \leftarrow 1$  to  $\delta$  do
14:     $\text{invpow} \leftarrow \text{invpow} \cdot \beta_i^{-1}$ 
15:     $N \leftarrow N \oplus \text{invpow} \cdot z_j$ 
16:  end for
17:  for all  $k \in \mathcal{E}$  do
18:    if  $k \neq i$  then
19:       $\beta_k \leftarrow \alpha^k$ 
20:       $D \leftarrow D \cdot (1 \oplus \beta_i^{-1} \beta_k)$ 
21:    end if
22:  end for
23:   $\mathbf{e}_{val}[i] \leftarrow N \cdot D^{-1}$ 
24: end for
25: return  $\mathbf{e}_{val}$ 

```
